playing Freeway. One will preprocess the states returned from the OpenAI Gym environment. We will get a 128-element numpy array with elements ranging from 0 to 255, and we'll need to convert it to a PyTorch tensor and normalize the values to be between 0 and 1 to moderate the size of the gradients.

We also need a policy function that decides which actions to take, given the predicted action-value distributions. With access to a full probability distribution over action values, we can utilize more sophisticated risk-sensitive policies. In this chapter, we will use a simple policy of choosing actions based on their expected value, in order to keep complexity to a minimum. Although we are learning a full probability distribution, we will choose actions based on their expected value, just like in ordinary Q-learning.

Listing 7.12. Preprocessing states and selecting actions

```
def preproc_state(state):
    p_state = torch.from_numpy(state).unsqueeze(dim=0).float()
    p_state = torch.nn.functional.normalize(p_state,dim=1)
1
    return p_state

def get_action(dist,support):
    actions = []
    for b in range(dist.shape[0]):
2
        expectations = [support @ dist[b,a,:] for a in
range(dist.shape[1])] 3
        action = int(np.argmax(expectations))
4
        actions.append(action)
    actions = torch.Tensor(actions).int()
    return actions
```

- ***1*** Normalizes state values to be between 0 and 1
- ***2*** Loops through batch dimension of distribution
- ***3*** Computes the expectation values for each action-value distribution
- ***4*** Computes the action associated with the highest expectation value

Recall, we can compute the expected (or expectation) value of a discrete distribution by simply taking the inner product of the support tensor with